Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 8

ПО КУРСУ

«Алгоритмические языки»

на тему «Файловая система. Сериализация»

Студент ИУ8-31 (Группа) Р. Д. Хан (И. О. Фамилия)

Преподаватель Р. Д. Исянов (И.О. Фамилия)

2026 г.

Цель работы

Изучить XML-сериализацию объектов и работу с файлами, каталогами, потоками и сжатием.

Условие задачи

1. На основе диаграммы ЛР7 реализовать сериализацию и десериализацию Animal. Инициализировать все его свойства, выполнить XML-сериализацию. В отдельном консольном проекте десериализовать объект и вывести его.

2. Найти файл по имени во всех поддиректориях пути, заданного пользователем. Вывести содержимое через FileStream. Добавить сжатие найденного файла стандартными средствами .NET.

Описание решения

Поскольку Animal абстрактный, переменная Animal содержит экземпляр Cow. XmlSerializer получает typeof(Animal), а XmlInclude перечисляет возможные производные типы. У конкретных классов есть открытые конструкторы без параметров; все свойства открыты для чтения и записи. Создание и чтение выполняются разными проектами.

Рекурсивный поиск проверяет исходный каталог и вложенные каталоги. Символические ссылки на каталоги пропускаются, чтобы не образовать цикл. Для текстового вывода FileStream передаётся StreamReader. GZipStream сжимает исходный поток в новый файл .gz. Исходный файл сохраняется, существующий архив не перезаписывается.

Порядок запуска

В Lab08Serialize сохранить animal.xml. В Lab08Deserialize указать тот же путь и прочитать объект. В Lab08Files ввести папку, имя файла и ответить «да» для сжатия.

Код программы на языке C#

Файл AnimalLibrary/Animal.cs

```
using System.Xml.Serialization;
namespace AnimalLibrary
{
    public enum eClassificationAnimal { Herbivores, Carnivores, Omnivores }
    public enum eFavoriteFood { Meat, Plants, Everything }

    [Comment("Общий абстрактный класс животного")]
    [XmlInclude(typeof(Cow))]
    [XmlInclude(typeof(Lion))]
    [XmlInclude(typeof(Pig))]
    public abstract class Animal
    {
        public string Country { get; set; }
        public bool HideFromOtherAnimals { get; set; }
        public string Name { get; set; }
        public string WhatAnimal { get; set; }
        public eClassificationAnimal Classification { get; set; }

        protected Animal() { }
```

```
        protected Animal(string name, string country, bool hide,
            string whatAnimal, eClassificationAnimal classification)
        {
            Name = name;
            Country = country;
            HideFromOtherAnimals = hide;
            WhatAnimal = whatAnimal;
            Classification = classification;
        }
        public void Deconstruct(out string name, out string country)
        {
            name = Name;
            country = Country;
        }
        public eClassificationAnimal GetClassificationAnimal()
        {
            return Classification;
        }
        public abstract eFavoriteFood GetFavoriteFood();
        public abstract string SayHello();
        public override string ToString()
        {
            return Name + ", " + WhatAnimal + ", " + Country
                + ", прячется: " + HideFromOtherAnimals
                + ", " + Classification + ", еда: " + GetFavoriteFood()
                + ", голос: " + SayHello();
        }
    }
}
```

Файл AnimalLibrary/CommentAttribute.cs

```
namespace AnimalLibrary
{
    [AttributeUsage(AttributeTargets.Class, Inherited = false)]
    [Comment("Пользовательский атрибут с комментарием")]
    public class CommentAttribute : Attribute
    {
        public string Comment { get; set; }
        public CommentAttribute(string comment)
        {
            Comment = comment;
        }
    }
}
```

Файл AnimalLibrary/Cow.cs

```
namespace AnimalLibrary
{
    [Comment("Корова")]
    public class Cow : Animal
    {
        public Cow() { }
        public Cow(string name, string country, bool hide)
            : base(name, country, hide, "Корова",
                eClassificationAnimal.Herbivores) { }
        public override eFavoriteFood GetFavoriteFood()
        {
            return eFavoriteFood.Plants;
        }
        public override string SayHello()
        {
            return "My!";
        }
    }
}
```

Файл AnimalLibrary/Lion.cs

```
namespace AnimalLibrary
{
    [Comment("Лев")]
```

```
    public class Lion : Animal
    {
        public Lion() { }
        public Lion(string name, string country, bool hide)
            : base(name, country, hide, "Лев",
                eClassificationAnimal.Carnivores) { }
        public override eFavoriteFood GetFavoriteFood()
        {
            return eFavoriteFood.Meat;
        }
        public override string SayHello()
        {
            return "P-p-p!";
        }
    }
}
```

Файл AnimalLibrary/Pig.cs

```
namespace AnimalLibrary
{
    [Comment("Свинья")]
    public class Pig : Animal
    {
        public Pig() { }
        public Pig(string name, string country, bool hide)
            : base(name, country, hide, "Свинья",
                eClassificationAnimal.Omnivores) { }
        public override eFavoriteFood GetFavoriteFood()
        {
            return eFavoriteFood.Everything;
        }
        public override string SayHello()
        {
            return "Хрю!";
        }
    }
}
```

Файл Lab08Serialize/Program.cs

```
using AnimalLibrary;
using System.Xml.Serialization;
namespace Lab08Serialize
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Animal animal = new Cow("Бурёнка", "Россия", false);
            string path = args.Length > 0 ? args[0] : "animal.xml";
            XmlSerializer serializer = new XmlSerializer(typeof(Animal));
            using (FileStream stream = new FileStream(path, FileMode.Create))
            {
                serializer.Serialize(stream, animal);
            }
            Console.WriteLine("До сериализации: " + animal);
            Console.WriteLine("Объект сохранён: " + Path.GetFullPath(path));
        }
    }
}
```

Файл Lab08Deserialize/Program.cs

```
using AnimalLibrary;
using System.Xml.Serialization;
namespace Lab08Deserialize
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
```

```
            string path = args.Length > 0 ? args[0] : "animal.xml";
            try
            {
                XmlSerializer serializer = new XmlSerializer(typeof(Animal));
                Animal animal;
                using (FileStream stream = new FileStream(path, FileMode.Open))
                {
                    animal = (Animal)serializer.Deserialize(stream);
                }
                Console.WriteLine("Тип после загрузки: " + animal.GetType().Name);
                Console.WriteLine("После десериализации: " + animal);
            }
            catch (Exception e)
            {
                Console.WriteLine("Не удалось прочитать XML: " + e.Message);
            }
        }
    }
}
```

Файл Lab08Files/Program.cs

```
using System.IO.Compression;
namespace Lab08Files
{
    internal class Program
    {
        static void Find(string folder, string fileName, List<string> result)
        {
            try
            {
                foreach (string file in Directory.GetFiles(folder))
                {
                    if (string.Equals(Path.GetFileName(file), fileName,
                        StringComparison.OrdinalIgnoreCase)) result.Add(file);
                }
                foreach (string dir in Directory.GetDirectories(folder))
                {
                    if ((File.GetAttributes(dir) & FileAttributes.ReparsePoint) != 0)
                        continue;
                    Find(dir, fileName, result);
                }
            }
            catch (UnauthorizedAccessException)
            {
                Console.WriteLine("Нет доступа: " + folder);
            }
            catch (IOException e)
            {
                Console.WriteLine("Ошибка чтения папки: " + e.Message);
            }
        }
        static void Main()
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Console.Write("Начальная папка: ");
            string folder = Console.ReadLine();
            Console.Write("Имя файла с расширением: ");
            string name = Console.ReadLine();
            if (!Directory.Exists(folder) || string.IsNullOrWhiteSpace(name))
            {
                Console.WriteLine("Неверная папка или имя.");
                return;
            }
            List<string> files = new List<string>();
            Find(folder, name, files);
            Console.WriteLine("Найдено файлов: " + files.Count);
            foreach (string file in files)
            {
                try
                {
                    Console.WriteLine("Файл: " + file);
```

```
                    using (FileStream stream = new FileStream(file, FileMode.Open, FileAccess.Read))
                    using (StreamReader reader = new StreamReader(stream))
                    {
                        Console.WriteLine(reader.ReadToEnd());
                    }
                    Console.Write("Сжать этот файл в GZip? (да/нет): ");
                    if (Console.ReadLine() != "да") continue;
                    string archive = file + ".gz";
                    if (File.Exists(archive))
                    {
                        Console.WriteLine("Архив уже существует: " + archive);
                        continue;
                    }
                    using (FileStream input = new FileStream(file, FileMode.Open, FileAccess.Read))
                    using (FileStream output = new FileStream(archive, FileMode.CreateNew))
                    using (GZipStream zip = new GZipStream(output, CompressionMode.Compress))
                    {
                        input.CopyTo(zip);
                    }
                    Console.WriteLine("Создан архив: " + archive);
                }
                catch (IOException e)
                {
                    Console.WriteLine("Ошибка файла: " + e.Message);
                }
                catch (UnauthorizedAccessException)
                {
                    Console.WriteLine("Нет доступа к файлу.");
                }
            }
        }
    }
}
```

Результаты выполнения программы

Объект Cow восстановлен через переменную Animal; тип, имя, страна, классификация, признак скрытности и остальные свойства сохранены. В тестовом вложенном каталоге найден note.txt. Созданный архив распакован при проверке: полученные байты полностью совпали с исходным файлом.

Lab08Serialize_run.txt

```
До сериализации: Бурёнка, Корова, Россия, прячется: False, Herbivores, еда: Plants, голос: Му!
Объект сохранён: /workspace/scratch/6823d4720338/compile_work/animal.xml
```

Lab08Deserialize_run.txt

```
Тип после загрузки: Cow
После десериализации: Бурёнка, Корова, Россия, прячется: False, Herbivores, еда: Plants, голос: Му!
```

Lab08Files_run.txt

```
Начальная папка: Имя файла с расширением: Найдено файлов: 1
Файл: /workspace/scratch/6823d4720338/compile_work/SearchDemo/subfolder/note.txt
Проверка FileStream и GZip.
Строка 2.

Сжать этот файл в GZip? (да/нет): Создан архив:
/workspace/scratch/6823d4720338/compile_work/SearchDemo/subfolder/note.txt.gz
```

Созданный XML включён в комплект: Results/animal.xml.

Выводы по работе

XML-сериализация и десериализация выполнены в отдельных приложениях с сохранением конкретного типа. Реализованы рекурсивный поиск, чтение через FileStream и сжатие GZip без изменения исходного файла.